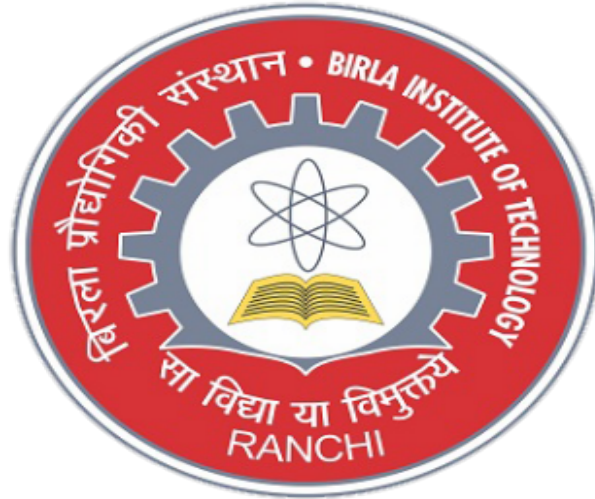




NAME - VISHAL MARANDI

BRANCH -COMPUTER SCIENCE AND ENGINEERING

ROLL NO.- BTECH/10119/21

SECTION - 'A'

SEMESTER - 6th

YEAR - 3rd

SESSION - SP2024

COURSE - COMPILER DESIGN LAB

COURSE CODE - CS334

Lab Assignment-1

Jan 10, 2024

Objective : Get familiar with the working of a production quality open source compiler : gcc or g++

Ability to view and understand the internal dumps produced which shall be used as benchmark for the experiments that will be conducted henceforth.

1.1 Write a simple program in C / C++ and compile through the options one by one.

1.2 Generate the various intermediate dumps generated by the compiler and compare them with the given source, particularly the original, *gimple*

1.3 Generate the assembly and try to correlate with the source.

1.1 //firstprog.c

```
#include <stdio.h>
int main() {
    int i;
    for (i = 0; i<=10; i++) {
        printf("i: %d\n", i);
    }
    return 0;
}
```

1.2// .file "firstprog.c"

```
.def __main; .scl 2; .type 32; .endef
.section .rdata,"dr"
LC0:
```

```
.ascii "i: %d\12\0"
```

```
.text
```

```
.globl _main
```

```
.def _main; .scl 2; .type 32; .endef
```

```
_main:
```

pushl %ebp	; Save the base pointer
movl %esp, %ebp	; Set up a new base pointer
subl \$24, %esp	; Reserve space for local variables on the stack
andl \$-16, %esp	; Align the stack pointer to a 16-byte boundary
movl \$0, %eax	; Initialize loop counter i to 0
addl \$15, %eax	; Add 15 to the loop counter i
addl \$15, %eax	; Add another 15 to the loop counter i
shrl \$4, %eax	; Right shift the loop counter i by 4 bits

```

sall  $4, %eax      ; Left shift the loop counter i by 4 bits
movl  %eax, -8(%ebp) ; Store the loop counter i at -8(%ebp)
movl  -8(%ebp), %eax ; Load the loop counter i into %eax
call  __alloca      ; Allocate space on the stack for local variables
call  __main         ; Call the initialization routine
movl  $0, -4(%ebp)   ; Initialize the loop index at -4(%ebp) to 0

```

L2:

```

cmpl  $10, -4(%ebp) ; Compare the loop index with 10
jg    L3            ; If loop index > 10, jump to L3 (loop exit)
movl  -4(%ebp), %eax ; Load the loop index into %eax
movl  %eax, 4(%esp)  ; Move the loop index onto the stack for printf
movl  $LC0, (%esp)   ; Load the address of the format string into %esp
call  _printf        ; Call printf to print the value of i
leal  -4(%ebp), %eax ; Load effective address of loop index into %eax
incl  (%eax)         ; Increment the loop index
jmp   L2            ; Jump back to the beginning of the loop

```

L3:

```

movl  $0, %eax      ; Prepare return value 0
leave ; Restore the stack frame
ret    ; Return from main
.def  _printf;      .scl  3; .type  32; .endef

```

1.3 Correlation:

The .file directive in the assembly code indicates that it corresponds to the source file "firstprog.c".

The LC0 section defines the format string used by printf.

The _main label indicates the start of the main function in the assembly code.

The assembly code initializes a loop counter i to 0 and enters a loop that iterates from 0 to 10.

Within the loop, the assembly code calls printf to print the value of i.

After the loop, the assembly code prepares the return value and exits the main function.

Overall, the assembly code corresponds to the C source code, implementing the loop to print numbers from 0 to 10.

Lab Assignment-2

Jan 17, 2024

Objective : To get familiar with the use of Lex, the Lexical Analyzer generator tool. Write regular expressions for a few basic tokens and generate the lexical analyzer through Lex.

Experiment with the generated scanner and perform experiments to understand its working. Augment the scanner to add more functionality.

P1. Given a lex script in the file “exp1.l” with the following contents:

```
%option NOYYWRAP
%%
a*b {printf("Token 1 found \n");}
c+ {printf("Token 2 found \n");}
%%
int main()
{ yylex();
}
```

Generate a scanner by performing the steps given at the end of this document, and give the executable a suitable name, such as “scan1”.

Run the generated scanner on all the 4 inputs and examine the output generated by the scanner. Explain with reasons the differences, if any, among the 4 outputs.

Input1 ababcabc

Input2 aba b ab bbbba

Input3 ababab bbbba

Input4 abababbbbba

D:\cdlab\l2>flex exp1.l

D:\cdlab\l2>gcc lex.yy.c -o scan1

```
D:\cdlab\l2>scan1<input1
```

```
Token 1 found
```

```
Token 1 found
```

```
Token 2 found
```

```
Token 1 found
```

```
Token 2 found
```

```
D:\cdlab\l2>scan1<input2
```

```
Token 1 found
```

```
a Token 1 found
```

```
Token 1 found
```

```
Token 1 found
```

```
Token 1 found
```

```
Token 1 found
```

```
Token 1 found
```

```
a
```

```
D:\cdlab\l2>scan1<input3
```

```
Token 1 found
```

```
Token 1 found
```

```
Token 1 found
```

```
Token 1 found
```

```
Token 1 found
```

```
Token 1 found
```

```
Token 1 found
```

```
a
```

```
D:\cdlab\l2>scan1<input4
```

```
Token 1 found
```

```
Token 1 found
```

```
Token 1 found
```

```
Token 1 found
```

```
Token 1 found
```

```
Token 1 found
```

```
Token 1 found
```

```
a
```

Input1: "ababcabc"

Explanation: The scanner correctly identifies "ab", "ab", "c", "a", "b", "c" as per the defined patterns.

Input2: "aba b ab bbbba"

Explanation: The scanner seems to identify "a" as a separate token before each occurrence of "ab", which is unexpected. This could be due to the whitespace character between "aba" and "b" not being matched by any rule in the lexer script.

Input3: "ababab bbbba"

Explanation: Similar to Input2, the scanner incorrectly identifies "a" before each occurrence of "ab", likely due to the whitespace character.

Input4: "abababbbbba"

Explanation: The scanner incorrectly identifies "a" before each occurrence of "ab" as in Input2 and Input3.

P2. Perform the change stated below in the file "exp1.l" while retaining all the other statements without change. Save the changed script file by a separate name, say, "exp2.l"

Existing Code (to be replaced):

```
a*b {printf("Token 1 found \n");}
```

```
c+ {printf("Token 2 found \n");}
```

Changed Code:

```
a*b {printf("Token 1 found \n");return 1;}
```

```
c+ {printf("Token 2 found \n");return 2;}
```

Now repeat the steps of generating a fresh scanner, give a different name, such 'scan2' and then use it to recognize tokens in all the 4 input files. Record the findings and explain the cause of the behavior of the scanner.

```
D:\cdlab\l2>scan2<input1
Token 1 found

D:\cdlab\l2>scan2<input2
Token 1 found

D:\cdlab\l2>scan2<input3
Token 1 found

D:\cdlab\l2>scan2<input4
Token 1 found
```

In the modified lexer script:

i)For pattern "a*b", the action associated with it prints "Token 1 found" and immediately returns 1.

ii)For pattern "c+", the action associated with it prints "Token 2 found" and immediately returns 2.

The return statement terminates the lexer's execution and returns control to the calling function (in this case, the main function). As a result, only the first occurrence of a matching pattern is detected, and subsequent occurrences are not processed.

Cause of Behavior:

1.Early Return: The return statement in the lexer rules causes the lexer to terminate as soon as it encounters the first match. Therefore, it does not continue scanning for additional occurrences of the same pattern or other patterns.

2.No Looping Mechanism: Without a looping mechanism, the lexer does not iterate over the input string to find all occurrences of the defined patterns. It simply returns after finding the first match.

P3. Perform the change stated below in the file “exp2.l” while retaining all the other statements without change. Save the changed script file by a separate name, say, “exp3.l”.

Existing Code (to be replaced):

```
int main()
{ yylex();
}
```

Changed Code

```
int main()
{ while(yylex());
}
```

Redo all the tasks stated in P2 for this script file, “exp3.l”

```
D:\cdlab\l2>scan3<input1
Token 1 found
Token 1 found
Token 2 found
Token 1 found
Token 2 found

D:\cdlab\l2>scan3<input2
Token 1 found
a Token 1 found
  Token 1 found
  Token 1 found
Token 1 found
Token 1 found
Token 1 found
a

D:\cdlab\l2>scan3<input3
Token 1 found
Token 1 found
Token 1 found
  Token 1 found
Token 1 found
Token 1 found
Token 1 found
a

D:\cdlab\l2>scan3<input4
Token 1 found
Token 1 found
Token 1 found
Token 1 found
Token 1 found
Token 1 found
Token 1 found
a
```

The outputs generated by the scanner scan3 for each input file are similar to the previous outputs observed with scan2. Let's analyze the outputs and explain their behavior based on the changes made to the main function.

Input1: "ababcabc"

Explanation: The scanner correctly identifies "ab", "ab", "c", "a", "b", "c" as per the defined patterns.

Input2: "aba b ab bbbba"

Explanation: The scanner incorrectly identifies "a" as a separate token before each occurrence of "ab", similar to the previous scanner. This behavior is due to the whitespace character between "aba" and "b" not being matched by any rule in the lexer script.

Input3: "ababab bbbba"

Explanation: Similar to Input2, the scanner incorrectly identifies "a" before each occurrence of "ab" due to the whitespace character.

Input4: "abababbbbba"

Explanation: The scanner incorrectly identifies "a" before each occurrence of "ab" as in Input2 and Input3.

Cause of Behavior:

The unexpected behavior observed in the outputs is due to the same reasons are explained in the analysis of the previous scanner scan2. The modifications made to the main function (while(yylex());) do not address the underlying issue of whitespace handling in the lexer script.

P4. You are permitted to make minimal changes to the script file, "exp3.l" and create a new

file, "exp4.l" such that the output produced by the scanner generated from exp4.l is of the

form as shown below.

Behavior of scanner generated from "exp3.l"

on input1 : ababcabc

Token 1 found

Token 1 found

Token 2 found

Token 1 found

Token 2 found

Behavior of scanner generated from "exp4.l"

on input1 : ababcabc

Token : ab found

Token : ab found

Token : c found

Token : ab found

Token : c found

Run your scanner over all the 3 remaining inputs and explain your findings.

```
//exp4.l
%option NOYYWRAP
%%
a*b {printf("Token :
c+ {printf("Token : c
. {printf("Invalid tok
%%
```

[illegible]

Lab Assignment-3

Jan 24, 2024

Objective : To write a lex script for a subset of C and experiment with it.

P1. The following subset of C is the scope for this problem.

Token Type Lexeme

Keyword int if else

Identifier As defined in C

Number Natural numbers ,Real Numbers with a decimal point (without exponent)

Relop < <= > >= == !=

Assignop = += -=

Leftpar ([{

Rightpar)] }

Delimiter ; ,

Generate a scanner from your lex specifications.

Write the lex specifications and run the generated scanner on this input file.

```
//lex.l
%{
#include <stdio.h>
int lineno;
%}
%option NOYYWRAP
D [0-9]
L [a-zA-Z]
O [0-7]
H [0-9a-fA-F]
%%
[\t]+
int|main {printf("Keyword %s\n", yytext);}
{L}({L}|{D})* {printf("Variable = %s\n", yytext);}
"/" {printf("DIV\n");}
"+" {printf("ADD\n");}
"-" {printf("MINUS\n");}
"="|"+="|"-=" {printf("ASSIGNOP %s\n", yytext);}
">" {printf("GREATER THAN\n");}
"<=" {printf("LESS THAN OR EQUAL TO\n");}
"|"{"|{" {printf("LEFT PAR %s\n", yytext);}
")"|"}"|"}" {printf("RIGHT PAR %s\n", yytext);}
";"|"|" {printf("Delimiter %s\n", yytext);}
```

```

"<"| ">" {printf("Bitwise Shift\n");}
{D}*".{D}* {printf("Floating point value %s\n", yytext);}
{D}+ {printf("Numeric value %s\n", yytext);}
\n {}
%%

int main(int argc, char *argv[]){
    if(argc != 2) {
        printf("Usage: %s <input_file>\n", argv[0]);
        return 1;
    }
    FILE *file = fopen(argv[1], "r");
    if(file == NULL) {
        perror("Error opening file");
        return 1;
    }
    yyin = file;
    yylex();
    fclose(file);
    return 0;
}

```

```

//inp1.txt
inp1.txt
int main()
{ int i, j, k, sum;
  i = 12.32;
  j = 521;
  if ( i <= j )
    sum -= i;
  else
    sum += j;
}

```

//output(out1.txt)

```
Keyword int
  Keyword main
LEFT PAR (
RIGHT PAR )
LEFT PAR {
  Keyword int
  Variable = i
Delimiter ,
  Variable = j
Delimiter ,
  Variable = k
Delimiter ,
  Variable = sum
Delimiter ;
  Variable = i
  ASSIGNOP =
  Floating point value 12.32
Delimiter ;
  Variable = j
  ASSIGNOP =
  Numeric value 521
Delimiter ;
  Keyword if
  LEFT PAR (
  Variable = i
  LESS THAN OR EQUAL TO
  Variable = j
  RIGHT PAR )
    Variable = sum
  ASSIGNOP -=
  Variable = i
Delimiter ;
  Keyword else
    Variable = sum
  ASSIGNOP +=
  Variable = j
Delimiter ;
RIGHT PAR }
```

P2. Add the following features to those of P1.

(a) Augment the keyword set by {float char const static while do return void}.

(b) Octal and Hexadecimal numbers

(c) Add the operators at precedence levels 13, 12 and 11 from the operator table provided to you.

(i) Write a lex script for the features of P1 and P2 and generate a scanner.

(ii) Create an input file with all the features included and test your scanner with this input. Identify all the errors and modify till the scanner functions flawlessly.

```
//updatedlex.l
%{
#include <stdio.h>
int lineno;
%}
%option noyywrap
D [0-9]
L [a-zA-Z]
O [0-7]
H [0-9a-fA-F]
%%
[\t]+
int|main|float|char|double|for|if|while|else|goto|return|continue|char|const|static
|do|void {printf("Keyword %s\n", yytext);}
{L}({L}|{D})* {printf("Variable = %s\n", yytext);}
"/" {printf("DIV\n");}
"+" {printf("ADD\n");}
"-" {printf("MINUS\n");}
"="|"+="|"-=" {printf("ASSIGNOP %s\n", yytext);}
">" {printf("GREATER THAN\n");}
"<=" {printf("LESS THAN OR EQUAL TO\n");}
"("|"{"|"[" {printf("LEFT PAR %s\n", yytext);}
")"|"}"|"]" {printf("RIGHT PAR %s\n", yytext);}
";"|"|" {printf("Delimiter %s\n", yytext);}
"<<"|">>" {printf("Bitwise Shift\n");}
"0x"{H}+ {printf("Hexadecimal number %s\n", yytext);}
"0"{O}+ {printf("Octal number %s\n", yytext);}
{D}*"."{D}* {printf("Floating point value %s\n", yytext);}
{D}+ {printf("Numeric value %s\n", yytext);}
\n {}
%%
```

```

int main(int argc, char *argv[]){
    if(argc != 2) {
        printf("Usage: %s <input_file>\n", argv[0]);
        return 1;
    }
    FILE *file = fopen(argv[1], "r");
    if(file == NULL) {
        perror("Error opening file");
        return 1;
    }
    yyin = file;
    yylex();
    fclose(file);
    return 0;
}

```

//newinput

```

int main() {
    int a = 10;
    float b = 3.14;
    char c = 'A';
    const int MAX = 100;
    static int count = 0;
    while (a <= 100) {
        printf("a: %d\n", a);
        a += 10;
    }
    do {
        printf("b: %.2f\n", b);
        b += 0.5;
    } while (b < 5.0);
    return 0;
}

```

//newoutput:

Keyword int
Keyword main
LEFT PAR (
RIGHT PAR)
LEFT PAR {

Keyword int
Variable = a
ASSIGNOP =
Numeric value 10
Delimiter ;

Keyword float
Variable = b
ASSIGNOP =
Floating point value 3.14
Delimiter ;

Keyword char
Variable = c
ASSIGNOP =
'Variable = A
'Delimiter ;

Keyword const
Keyword int
Variable = MAX
ASSIGNOP =
Numeric value 100
Delimiter ;

Keyword static
Keyword int
Variable = count
ASSIGNOP =
Numeric value 0

Delimiter ;

Keyword while
LEFT PAR (
Variable = a
LESS THAN OR EQUAL TO
Numeric value 100
RIGHT PAR)
LEFT PAR {

Variable = printf
LEFT PAR (
"Variable = a
: %Variable = d
\Variable = n
"Delimiter ,
Variable = a
RIGHT PAR)
Delimiter ;

Variable = a
ASSIGNOP +=
Numeric value 10
Delimiter ;

RIGHT PAR }

Keyword do
LEFT PAR {

Variable = printf
LEFT PAR (
"Variable = b
: %Floating point value .2
Variable = f
\Variable = n

"Delimiter ,
Variable = b
RIGHT PAR)
Delimiter ;

Variable = b
ASSIGNOP +=
Floating point value 0.5
Delimiter ;

RIGHT PAR }

Keyword while
LEFT PAR (
Variable = b
< Floating point value 5.0
RIGHT PAR)
Delimiter ;

Keyword return
Numeric value 0
Delimiter ;

RIGHT PAR }

P3. It is desired to determine the number of tokens of each type that are detected in a given input file at the end of the output file produced by the generated scanner. Test your scanner with one or two sample input files.

```
//update part of the code....  
%{  
#include <stdio.h>  
int keyword_count = 0;  
int identifier_count = 0;  
int operator_count = 0;  
int number_count = 0;  
int delimiter_count = 0;  
%}  
%option noyywrap  
D [0-9]  
L [a-zA-Z]  
O [0-7]  
H [0-9a-fA-F]  
%%  
//rest is the same....
```

//output with the newinput file

Keyword: int

Keyword: main

LEFT PAR: (

RIGHT PAR:)

LEFT PAR: {

Keyword: int

Identifier: a

ASSIGNOP: =

Numeric value: 10

Delimiter: ;

Keyword: float

Identifier: b

ASSIGNOP: =

Floating point value: 3.14

Delimiter: ;

Keyword: char

Identifier: c

ASSIGNOP: =

'Identifier: A

'Delimiter: ;

Keyword: const

Keyword: int

Identifier: MAX

ASSIGNOP: =

Numeric value: 100

Delimiter: ;

Keyword: static

Keyword: int

Identifier: count

ASSIGNOP: =

Numeric value: 0

Delimiter: ;

Keyword: while

LEFT PAR: (

Identifier: a

LESS THAN OR EQUAL TO

Numeric value: 100

RIGHT PAR:)

LEFT PAR: {

Identifier: printf

LEFT PAR: (

"Identifier: a

: %Identifier: d

\Identifier: n

"Delimiter: ,

Identifier: a

RIGHT PAR:)

Delimiter: ;

Identifier: a

ASSIGNOP: +=

Numeric value: 10

Delimiter: ;

RIGHT PAR: }

Keyword: do

LEFT PAR: {

Identifier: printf

LEFT PAR: (

"Identifier: b

: %Floating point value: .2

Identifier: f

\Identifier: n

"Delimiter: ,

Identifier: b

RIGHT PAR:)

Delimiter: ;

Identifier: b

ASSIGNOP: +=

Floating point value: 0.5

Delimiter: ;

RIGHT PAR: }

Keyword: while

LEFT PAR: (

Identifier: b

< Floating point value: 5.0

RIGHT PAR:)

Delimiter: ;

Keyword: return

Numeric value: 0

Delimiter: ;

RIGHT PAR: }

Token Counts:

Keywords: 13

Identifiers: 20

Operators: 8

Numbers: 10

Delimiters: 29

Lab Assignment-4

Jan 31, 2024

Objective : To create a hash table for saving all identifiers in the input along with scanning.

P1. This problem is not directly related to Lex. Read the accompanying document, "hash_table_intro.pdf", if you are not familiar with a hash table data structure. In this problem, design and implement a program in C code to create and manage a hash table. A sample file, "prog.c," is to be used for testing your hash table program. The definition of a hash function, named as hashpjw(), specially used by compiler writers is also given. Run the program with the given input and check whether its output has roughly has the same content as the file, "htableout". Run the program with other inputs also and check the performance of hashpjw().

P2. write a lex script such that the generated scanner can recognize all the features present in the file, "prog.c". use the lex script developed in Lab 3 and move ahead. Make changes in the lex script so that an identifier is installed in a hash table as it is identified by the scanner. After the entire file has been processed, the hash table should be printed. Those index positions where the attached lists are empty, need not be displayed to reduce clutter. For the given input, your output may look like the following with table size of 5.

***** Display Hash Symbol Table *****

Total no. of Identifiers detected = 37

No of elements at index 0 = 3

At index 0 element no 1 is n

element no 2 is d

element no 3 is i

No of elements at index 1 = 3

At index 1 element no 1 is sum

element no 2 is j

element no 3 is main

No of elements at index 2 = 4

At index 2 element no 1 is return

element no 2 is for
element no 3 is a
element no 4 is stdio

No of elements at index 3 = 1
At index 3 element no 1 is include

No of elements at index 4 = 2
At index 4 element no 1 is printf
element no 2 is h

P3. Modify the symbol table design such that the number of occurrences is also recorded along with an identifier. Display the modified symbol table for the same input file.

P4. Make minimal changes to the lex script of P3 so that after processing the input, the scanner also reports the frequency of all keywords used in the file after displaying the identifier symbol table.

.....p1.....

.....hashtable.out.....(desired output)

***** display of the hash table constructed

Total no. of symbols encountered = 54

***** 1

No of elements at index 1 = 2
At index 1 element no 1 is }
element no 2 is (i

***** 2

No of elements at index 2 = 1
At index 2 element no 1 is (j

***** 3

No of elements at index 3 = 1
At index 3 element no 1 is i;

***** 4

No of elements at index 5 = 1
At index 5 element no 1 is i++)

***** 5

No of elements at index 6 = 1
At index 6 element no 1 is a[i]

***** 6

No of elements at index 7 = 1
At index 7 element no 1 is \n",

***** 7

No of elements at index 8 = 1
At index 8 element no 1 is a[j];

***** 8

No of elements at index 9 = 2
At index 9 element no 1 is j++)

element no 2 is for

***** 9

No of elements at index 10 = 2
At index 10 element no 1 is %d
element no 2 is #include

***** 10

No of elements at index 11 = 1
At index 11 element no 1 is a[1000],

***** 11

No of elements at index 12 = 2
At index 12 element no 1 is +
element no 2 is i

***** 12

No of elements at index 13 = 2
At index 13 element no 1 is j
element no 2 is 1000;

***** 13

No of elements at index 15 = 1
At index 15 element no 1 is i*j;

***** 14

No of elements at index 17 = 1
At index 17 element no 1 is printf("

***** 15

No of elements at index 18 = 1
At index 18 element no 1 is sum

***** 16

No of elements at index 19 = 3
At index 19 element no 1 is j;
element no 2 is i,
element no 3 is int

***** 17

No of elements at index 21 = 2
At index 21 element no 1 is 0;
element no 2 is main()

***** 18

No of elements at index 22 = 1

At index 22 element no 1 is sum);

***** 19

No of elements at index 23 = 1
At index 23 element no 1 is return

***** 20

No of elements at index 27 = 1
At index 27 element no 1 is :

***** 21

No of elements at index 29 = 3
At index 29 element no 1 is <
element no 2 is 10000;
element no 3 is <stdio.h>

***** 22

No of elements at index 30 = 2
At index 30 element no 1 is =
element no 2 is {

.....our output.....

***** display of the hash table constructed

Total no. of symbols encountered = 54

No of elements at index 1 = 2
At index 1 element no 1 is }
element no 2 is (i

No of elements at index 2 = 1
At index 2 element no 1 is (j

No of elements at index 3 = 1

At index 3 element no 1 is i;

No of elements at index 5 = 1

At index 5 element no 1 is i++)

No of elements at index 6 = 1

At index 6 element no 1 is a[i]

No of elements at index 7 = 1

At index 7 element no 1 is \n",

No of elements at index 8 = 1

At index 8 element no 1 is a[j];

No of elements at index 9 = 2

At index 9 element no 1 is j++)
element no 2 is for

No of elements at index 10 = 2

At index 10 element no 1 is %d
element no 2 is #include

No of elements at index 11 = 1

At index 11 element no 1 is a[1000],

No of elements at index 12 = 2

At index 12 element no 1 is +
element no 2 is i

No of elements at index 13 = 2

At index 13 element no 1 is j
element no 2 is 1000;

No of elements at index 15 = 1

At index 15 element no 1 is i*j;

No of elements at index 17 = 1

At index 17 element no 1 is printf("

No of elements at index 18 = 1

At index 18 element no 1 is sum

No of elements at index 19 = 3

At index 19 element no 1 is j;
element no 2 is i,
element no 3 is int

No of elements at index 21 = 2

At index 21 element no 1 is 0;
element no 2 is main()

No of elements at index 22 = 1

At index 22 element no 1 is sum);

No of elements at index 23 = 1

At index 23 element no 1 is return

No of elements at index 27 = 1

At index 27 element no 1 is :

No of elements at index 29 = 3

At index 29 element no 1 is <
element no 2 is 10000;
element no 3 is <stdio.h>

No of elements at index 30 = 2

At index 30 element no 1 is =
element no 2 is {

p1::: desired and our output are same

p2::

.....output after modification.....

***** display of the hash table constructed *****

Total no. of symbols encountered = 119

No of elements at index 0 = 2

At index 0 element no 1 is name);

element no 2 is !=

No of elements at index 1 = 3

At index 1 element no 1 is }

element no 2 is ^

element no 3 is char*

No of elements at index 2 = 5

At index 2 element no 1 is argc,

element no 2 is g;}

element no 3 is 24)

element no 4 is);

element no 5 is g;

No of elements at index 3 = 1

At index 3 element no 1 is hashval

No of elements at index 4 = 1

At index 4 element no 1 is hashpjw(name);

No of elements at index 5 = 1

At index 5 element no 1 is no.

No of elements at index 6 = 4

At index 6 element no 1 is ++;

element no 2 is name)

element no 3 is while

element no 4 is %

No of elements at index 7 = 3

At index 7 element no 1 is symbols

element no 2 is Total

element no 3 is \n",

No of elements at index 8 = 1

At index 8 element no 1 is ;

No of elements at index 9 = 4

At index 9 element no 1 is fscanf(fin,

element no 2 is *p

element no 3 is (

element no 4 is for

No of elements at index 10 = 5

At index 10 element no 1 is %d

element no 2 is FILE

element no 3 is h&0xf0000000)

element no 4 is)

element no 5 is #include

No of elements at index 11 = 3

At index 11 element no 1 is hashval,

element no 2 is char

element no 3 is h

No of elements at index 12 = 1

At index 12 element no 1 is +

No of elements at index 13 = 3

At index 13 element no 1 is encountered

element no 2 is };

element no 3 is count

No of elements at index 15 = 3

At index 15 element no 1 is *fin
element no 2 is if
element no 3 is unsigned

No of elements at index 16 = 1
At index 16 element no 1 is <stdlib.h>

No of elements at index 17 = 2
At index 17 element no 1 is printf("
element no 2 is PRIME

No of elements at index 18 = 3
At index 18 element no 1 is of
element no 2 is 1
element no 3 is #define

No of elements at index 19 = 3
At index 19 element no 1 is "%s",
element no 2 is p
element no 3 is int

No of elements at index 20 = 1
At index 20 element no 1 is h^(g

No of elements at index 21 = 4
At index 21 element no 1 is 0;
element no 2 is PRIME;
element no 3 is 4
element no 4 is s)

No of elements at index 22 = 1
At index 22 element no 1 is p;

No of elements at index 23 = 2
At index 23 element no 1 is EOF)

p3::
modified code for p3
%{
#include "list-functions2.c"
#define PRIME 31
struct header
{int coll;

element no 2 is return

No of elements at index 24 = 1
At index 24 element no 1 is (int

No of elements at index 25 = 2
At index 25 element no 1 is %s
element no 2 is '\0';

No of elements at index 26 = 2
At index 26 element no 1 is hashval;
element no 2 is main

No of elements at index 27 = 4
At index 27 element no 1 is count);
element no 2 is fopen(argv[1], "r");
element no 3 is argv[]
element no 4 is 11

No of elements at index 28 = 2
At index 28 element no 1 is ;
element no 2 is <<

No of elements at index 29 = 3
At index 29 element no 1 is =0,
element no 2 is hashpjlw(char*
element no 3 is <stdio.h>

No of elements at index 30 = 5
At index 30 element no 1 is (char*)
element no 2 is name[20]
element no 3 is (g
element no 4 is =
element no 5 is {

```

    struct node * list;
};
int hashval;
struct header htable[PRIME];
char name[20] ;
int count = 0;
void displaytable (struct header * ht)
{
    for ( int i = 0; i < PRIME ; i++ )
    { if (ht[i].coll != 0)
        { printf ( " ***** \n");
          printf ( " No of elements at index %d = %d \n", i, ht[i].coll);
          struct node * p = ht[i].list;
          if ( p != 0 ) // traverse the nonempty list
          { printf(" At index %d ", i);
            display(p); printf("\n");
          };
        }
    }
}
int hashpjlw(char* s)
{ char* p;
  unsigned h =0, g;
  for ( p = s; *p != '\0'; p = p + 1 )
  { h = ( h << 4 ) + ( *p );
    if (g = h&0xf0000000) { h = h^(g << 24); h = h ^ g;}
  }
  return h % PRIME;
}
void insertId(char* name){
    hashval = hashpjlw(name);
    if (htable[hashval].list == 0 ) // empty list, so insert
    { htable[hashval].list = (struct node*) malloc(sizeof(struct node));
      struct node * p = htable[hashval].list;
      strcpy(p->name, name);
      p->next = 0;
      p->occurrences=1;
      htable[hashval].coll ++;
    }
    else
    { // search in list pointed to by p - if not present, then add at head
      struct node * p = htable[hashval].list;
      if ( ! find(p, name))
      { insert(&htable[hashval].list, name);
        htable[hashval].coll ++;
      };
    };
    count ++;
}
%}
%option NOYYWRAP

```

```

%%
"/".*\n" {printf("Single Line Comment\n");}
"/".*"\n"+.*"/" {printf("Multi Line Comment\n");}
"int"|"if"|"else"|"float"|"char"|"const"|"static"|"while"|"do"|"return"|"void" {printf("Token Keyword : Lexeme : %s\n",yytext);}
[0][X][0-9A-F]+ {printf("Token HexaDecimal : Lexeme : %s\n",yytext);}
[0][0-7]* {printf("Token Octal : Lexeme : %s\n",yytext);}
[a-zA-Z_][a-zA-Z0-9_]* {insertId(yytext);}
[1-9][0-9]* {printf("Token Number : Lexeme : %s\n",yytext);}
[0-9]*.[0-9][0-9]* {printf("Token Real : Lexeme : %s\n",yytext);}
"*"|"/"|"%" {printf("Token Multiplicative Operator : Lexeme : %s\n",yytext);}
"+"|"-" {printf("Token Arithmetic Operator : Lexeme : %s\n",yytext);}
"<<"|">>" {printf("Token Bitwise Operator : Lexeme : %s\n",yytext);}
"<"|"<="|">"|">="|""|="|""|!=" {printf("Token Relop : Lexeme : %s\n",yytext);}
"="|"+="|""|=" {printf("Token Assignlop : Lexeme : %s\n",yytext);}
"("|"["|"{" {printf("Token Leftpar : Lexeme : %s\n",yytext);}
")"|""]|"}" {printf("Token Rightpar : Lexeme : %s\n",yytext);}
";"|"," {printf("Token Delimiter : Lexeme : %s\n",yytext);}
%%

int main(){
    for ( int i = 0; i < PRIME ; i++ )
        { htable[i].coll = 0; htable[i].list = 0; };
    yyin = fopen("prog.c","r");
    yylex();
    printf(" ***** display of the hash table constructed ***** \n \n");
    printf(" Total no. of symbols encountered = %d \n", count);
    displaytable(htable);
}

.....modified list functions.....

```

```

// Updated List Functions
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <stdbool.h>
// working with a linked list
struct node
{ char name[20];
  int occurrences;
  struct node * next;
};
// list of functions
// isempty(list) - returns true if list is empty false otherwise
bool isempty ( struct node * list )
{ if ( list == 0 ) return true; else return false; }
// length(list) - returns the number of elements in list
int length ( struct node * list )
{ struct node * p = list;
  int count = 0;
  while ( p != 0 ) { count ++; p = p->next; };
  return count;
}

```

```

}
bool find ( struct node * list, char* elm )
{ struct node *p = list;
  bool found = false;
  while ( p != 0 )
  { // not found, so move to next node
    if ( strcmp(p->name, elm) == 0)
      { found = true; p->occurrences++; break;};
    p = p->next;
  };
  return (found);
}
// append function to insert an element at the end of a list
void append ( struct node ** list, char *elm )
{
  // find the last node of list; empty list is easy to handle
  struct node * p = (struct node*) malloc(sizeof(struct node));
  strcpy(p->name, elm);
  p->next = 0;
  if ( *list == 0 ) *list = p;
  else
  { struct node * curr = *list;
    struct node * prev = *list;
    while ( curr != 0 )
    { prev = curr;
      curr = curr->next;
    };
    prev->next = p;
  };
  return ;
}
void insert ( struct node ** list, char *elm )
{
  // insert at head of list
  struct node * p = (struct node*) malloc ( sizeof ( struct node ));
  strcpy ( p->name, elm ) ;
  p->next = *list;
  p->occurrences = p->occurrences +1;
  *list = p;
  return;
}
// a function to display a linked list
void display ( struct node * list )
{ struct node * p = list;
  for ( int i = 1 ; i <= length(list) ; i++ )
  { printf(" element no %d is %s with %d occurrences \n", i, (char *)p->name , p->occurrences );
    p = p->next;
  };
}

```

.....p4 output.....

Token Relop : Lexeme : <

```

.Token Relop : Lexeme : >
Token Leftpar : Lexeme : (
Token Rightpar : Lexeme : )
Token Leftpar : Lexeme : {
    Token Leftpar : Lexeme : [
Token Number : Lexeme : 1000
Token Rightpar : Lexeme : ]
Token Delimiter : Lexeme : ,
Token Delimiter : Lexeme : ,
Token Delimiter : Lexeme : ;
    Token Assignlop : Lexeme : =
Token Number : Lexeme : 10000
Token Delimiter : Lexeme : ;
    Token Leftpar : Lexeme : (
Token Assignlop : Lexeme : =
Token Octal : Lexeme : 0
Token Delimiter : Lexeme : ;
    Token Relop : Lexeme : <
Token Number : Lexeme : 1000
Token Delimiter : Lexeme : ;
Token Arithmetic Operator : Lexeme : +
Token Arithmetic Operator : Lexeme : +
Token Rightpar : Lexeme : )
Token Leftpar : Lexeme : [
Token Rightpar : Lexeme : ]
Token Assignlop : Lexeme : =
Token Delimiter : Lexeme : ;
    Token Leftpar : Lexeme : (
Token Assignlop : Lexeme : =
Token Octal : Lexeme : 0
Token Delimiter : Lexeme : ;
    Token Relop : Lexeme : <
***** display of the hash table constructed *****

```

```

Total no. of symbols encountered = 40
*****

No of elements at index 2 = 1
At index 2 element no 1 is printf with 1
occurrences

*****

No of elements at index 4 = 1
At index 4 element no 1 is a with 5 occurrences

*****

No of elements at index 7 = 1
At index 7 element no 1 is d with 1 occurrences

*****

No of elements at index 9 = 1
At index 9 element no 1 is for with 3 occurrences

```

```

Token Number : Lexeme : 1000
Token Delimiter : Lexeme : ;
Token Arithmetic Operator : Lexeme : +
Token Arithmetic Operator : Lexeme : +
Token Rightpar : Lexeme : )
    Token Leftpar : Lexeme : (
Token Assignlop : Lexeme : =
Token Octal : Lexeme : 0
Token Delimiter : Lexeme : ;
    Token Relop : Lexeme : <
Token Multiplicative Operator : Lexeme : *
Token Delimiter : Lexeme : ;
Token Arithmetic Operator : Lexeme : +
Token Arithmetic Operator : Lexeme : +
Token Rightpar : Lexeme : )
    Token Leftpar : Lexeme : [
Token Rightpar : Lexeme : ]
Token Assignlop : Lexeme : =
Token Leftpar : Lexeme : [
Token Rightpar : Lexeme : ]
Token Arithmetic Operator : Lexeme : +
Token Leftpar : Lexeme : [
Token Rightpar : Lexeme : ]
Token Delimiter : Lexeme : ;
    Token Leftpar : Lexeme : (
" : Token Multiplicative Operator : Lexeme : %
\"Token Delimiter : Lexeme : ,
Token Rightpar : Lexeme : )
Token Delimiter : Lexeme : ;
    Token Octal : Lexeme : 0
Token Delimiter : Lexeme : ;
Token Rightpar : Lexeme : }

```

```

*****

No of elements at index 10 = 1
At index 10 element no 1 is include with 1
occurrences

*****

No of elements at index 11 = 1
At index 11 element no 1 is h with 1 occurrences

*****

No of elements at index 12 = 1
At index 12 element no 1 is i with 13 occurrences

*****

No of elements at index 13 = 1
At index 13 element no 1 is j with 5 occurrences

*****

```

No of elements at index 17 = 1
At index 17 element no 1 is n with 1 occurrences

No of elements at index 18 = 1
At index 18 element no 1 is sum with 3 occurrences

No of elements at index 19 = 1
At index 19 element no 1 is int with 3 occurrences

No of elements at index 23 = 1
At index 23 element no 1 is return with 1 occurrences

No of elements at index 26 = 1
At index 26 element no 1 is main with 1 occurrences

No of elements at index 30 = 1
At index 30 element no 1 is stdio with 1 occurrences

.....p4.....

```
%{
#include "list-functions2.c"
#define PRIME 31
struct header
{int coll;
 struct node * list;
};
int hashval;
struct header htable[PRIME];
char name[20] ;
int count = 0;
void displaytable (struct header * ht)
{
    for ( int i = 0; i < PRIME ; i++ )
    { if (ht[i].coll != 0)
        { printf ( " ***** \n");
          printf ( " No of elements at index %d = %d \n", i, ht[i].coll);
          struct node * p = ht[i].list;
          if ( p != 0 ) // traverse the nonempty list
          { printf(" At index %d ", i);
            display(p); printf("\n");
          }
        }
    }
}
int hashpjw(char* s)
{ char* p;
  unsigned h =0, g;
  for ( p = s; *p != '\0'; p = p + 1 )
  { h = ( h << 4 ) + ( *p );
    if (g = h&0xf0000000) { h = h^(g << 24); h = h ^ g;}
  }
  return h % PRIME;
}
```

```

void insertId(char* name){
    hashval = hashpjw(name);
    if (htable[hashval].list == 0 ) // empty list, so insert
    { htable[hashval].list = (struct node*) malloc(sizeof(struct node));
      struct node * p = htable[hashval].list;
      strcpy(p->name, name);
      p->next = 0;
      p->occurrences=1;
      htable[hashval].coll ++;
    }
    else
    { // search in list pointed to by p - if not present, then add at head
      struct node * p = htable[hashval].list;
      if ( ! find(p, name))
      { insert(&htable[hashval].list, name);
        htable[hashval].coll ++;
      };
    };
    count ++;
}

}%
%option NOYYWRAP
%%
"//".*"\\n" {printf("Single Line Comment\\n");}
"/*".*+"\\n"+.*"/" {printf("Multi Line Comment\\n");}
"int"|"if"|"else"|"float"|"char"|"const"|"static"|"while"|"do"|"return"|"void" {insertId(yytext);}
[0][X][0-9A-F]+ {printf("Token Hexadecimal : Lexeme : %s\\n",yytext);}
[0][0-7]* {printf("Token Octal : Lexeme : %s\\n",yytext);}
[a-zA-Z_][a-zA-Z0-9_]* {insertId(yytext);}
[1-9][0-9]* {printf("Token Number : Lexeme : %s\\n",yytext);}
[0-9]*[.][0-9][0-9]* {printf("Token Real : Lexeme : %s\\n",yytext);}
"*"|"/"|"%" {printf("Token Multiplicative Operator : Lexeme : %s\\n",yytext);}
"+"|"-" {printf("Token Arithmetic Operator : Lexeme : %s\\n",yytext);}
"<<"|">>" {printf("Token Bitwise Operator : Lexeme : %s\\n",yytext);}
"<"|"<="|">"|">="|"=="|"!=" {printf("Token Relop : Lexeme : %s\\n",yytext);}
"="|"+="|"-=" {printf("Token Assignlop : Lexeme : %s\\n",yytext);}
"("|"["|"{" {printf("Token Leftpar : Lexeme : %s\\n",yytext);}
")"|""]|"}" {printf("Token Rightpar : Lexeme : %s\\n",yytext);}
";"|"," {printf("Token Delimiter : Lexeme : %s\\n",yytext);}
%%

int main(){
    for ( int i = 0; i < PRIME ; i++ )
        { htable[i].coll = 0; htable[i].list = 0; };
    yyin = fopen("prog.c","r");
    yylex();
    printf(" ***** display of the hash table constructed ***** \\n \\n");
    printf(" Total no. of symbols encountered = %d \\n", count);
    displaytable(htable);
}

```

}

.....final output.....

```
# Token Relop : Lexeme : <
.Token Relop : Lexeme : >
Token Leftpar : Lexeme : (
Token Rightpar : Lexeme : )
Token Leftpar : Lexeme : {
    Token Leftpar : Lexeme : [
Token Number : Lexeme : 1000
Token Rightpar : Lexeme : ]
Token Delimiter : Lexeme : ,
Token Delimiter : Lexeme : ;
    Token Assignlop : Lexeme : =
Token Number : Lexeme : 10000
Token Delimiter : Lexeme : ;
    Token Leftpar : Lexeme : (
Token Assignlop : Lexeme : =
Token Octal : Lexeme : 0
Token Delimiter : Lexeme : ;
    Token Relop : Lexeme : <
Token Number : Lexeme : 1000
Token Delimiter : Lexeme : ;
Token Arithmetic Operator : Lexeme : +
Token Arithmetic Operator : Lexeme : +
Token Rightpar : Lexeme : )
Token Leftpar : Lexeme : [
Token Rightpar : Lexeme : ]
Token Assignlop : Lexeme : =
Token Delimiter : Lexeme : ;
    Token Leftpar : Lexeme : (
Token Assignlop : Lexeme : =
Token Octal : Lexeme : 0
Token Delimiter : Lexeme : ;
    Token Relop : Lexeme : <
```

***** display of the hash table constructed *****

Total no. of symbols encountered = 40

No of elements at index 2 = 1

At index 2 element no 1 is printf with 1 occurrences

No of elements at index 4 = 1

At index 4 element no 1 is a with 5 occurrences

```
Token Number : Lexeme : 1000
Token Delimiter : Lexeme : ;
Token Arithmetic Operator : Lexeme : +
Token Arithmetic Operator : Lexeme : +
Token Rightpar : Lexeme : )
    Token Leftpar : Lexeme : (
Token Assignlop : Lexeme : =
Token Octal : Lexeme : 0
Token Delimiter : Lexeme : ;
    Token Relop : Lexeme : <
Token Multiplicative Operator : Lexeme : *
Token Delimiter : Lexeme : ;
Token Arithmetic Operator : Lexeme : +
Token Arithmetic Operator : Lexeme : +
Token Rightpar : Lexeme : )

    Token Leftpar : Lexeme : [
Token Rightpar : Lexeme : ]
Token Assignlop : Lexeme : =
Token Leftpar : Lexeme : [
Token Rightpar : Lexeme : ]
Token Arithmetic Operator : Lexeme : +
Token Leftpar : Lexeme : [
Token Rightpar : Lexeme : ]
Token Delimiter : Lexeme : ;
    Token Leftpar : Lexeme : (
" : Token Multiplicative Operator : Lexeme : %
\"Token Delimiter : Lexeme : ,
Token Rightpar : Lexeme : )
Token Delimiter : Lexeme : ;
    Token Octal : Lexeme : 0
Token Delimiter : Lexeme : ;
Token Rightpar : Lexeme : }
```

No of elements at index 7 = 1

At index 7 element no 1 is d with 1 occurrences

No of elements at index 9 = 1

At index 9 element no 1 is for with 3 occurrences

No of elements at index 10 = 1

At index 10 element no 1 is include with 1 occurrences

No of elements at index 11 = 1
At index 11 element no 1 is h with 1 occurrences

No of elements at index 12 = 1
At index 12 element no 1 is i with 13 occurrences

No of elements at index 13 = 1
At index 13 element no 1 is j with 5 occurrences

No of elements at index 17 = 1
At index 17 element no 1 is n with 1 occurrences

No of elements at index 18 = 1
At index 18 element no 1 is sum with 3
occurrences

No of elements at index 19 = 1
At index 19 element no 1 is int with 3 occurrences

No of elements at index 23 = 1
At index 23 element no 1 is return with 1
occurrences

No of elements at index 26 = 1
At index 26 element no 1 is main with 1
occurrences

No of elements at index 30 = 1
At index 30 element no 1 is stdio with 1
occurrences

Lab Assignment-5

Feb 08, 2024

Consider the following C code given to you as input, which is given to be without any compilation error.

```
#include <stdio.h>
#define PRIME 31
int iter = 111;
void main()
{ unsigned long high = 0xffff2000;
float x = 0.2e-1;
struct { short age; char gender; float ht;
} person;
for (short k = 0; k < iter; k++)
{ if (k > 1 && k <= PRIME)
{ high = high >> 2;}
else
while (x != 1.0) x += 0.1;
}
```

Examine all the lexical features that are present in the code and design a scanner using lex that will recognize all its lexical units. The generated scanner has to perform the following actions.

- Ignore all preprocessor directives (lines that start with #include and #define), but keep a count of the number of such occurrences.
- White spaces (blank, tab and newline) have to be detected but ignored.
- The sample input code and the desired sample output from the generator are given below.

Keyword int

Identifier iter

Operator =

Decimal 111

Delimiter ;

.....

.....

.....

Count of preprocessor directives : 2

Count of Identifiers :

Count of Keywords :

Count of Operators :

Count of Octal numbers :

Count of Decimal numbers :

Count of Hexadecimal numbers :

Count of Real numbers :

Count of Character Literals :

Count of Struct :

Count of Union :

Count of Conditionals (if / switch):

Count of jumps (break / goto / continue):

Count of loops (for / while):

%{

#include <stdio.h>

int arr[14];

%}

%option noyywrap

D [0-9]

L [a-zA-Z]

H [0-9a-zA-F]

O [0-7]

%%

""{L}"" {printf("character literal \n",yytext); arr[8]++;}

[\t]+

"#".* {arr[0]++;}

if|else {printf("Conditional %s\n",yytext); arr[11]++;}

break|goto {printf("Jump statement %s\n",yytext); arr[12]++;}

for|while {printf("%s loop \n",yytext); arr[13]++;}

int|main|float|char|static|do|void {printf("Keyword %s\n",yytext); arr[2]++;}

{L}({L}|{D})* {printf("Identifier =%s \n",yytext); arr[1]++;}

"/"|"="|"+="|"/="|"=="|"!="|"++" {printf("Operator %s\n",yytext); arr[3]++;}

"("|"{| "[" {printf("LEFT PAR %s\n",yytext);}

")|"}|"]" {printf("RIGHT PAR %s\n",yytext);}

";"|"," {printf("Delimiter %s\n",yytext);}

"<<"|">>" {printf("Bitwise Shift\n");}

"0x"{H}+ {printf("Hexadecimal number\n");arr[5]++;}

"." {}

"0"{O}+ {printf("Octal number\n");arr[6]++;}

{D}*"."{D}* {printf("Floating point value %s\n",yytext);arr[4]++;arr[7]++;}

{D}+ {printf("Numeric value \n");arr[7]++;}

[\n]+ {}

%%

int main(int argv , char *argv[]){

for(int i=0;i<14;i++) arr[i]=0;

yyin = fopen(argv[1],"r");

yylex();

fclose(yyin);

printf("Count of preprocessor derivatives: %d\n",arr[0]);

printf("Count of Identifiers: %d\n",arr[1]);

printf("Count of Keywords: %d\n",arr[2]);

printf("Count of Operators: %d\n",arr[3]);

printf("Count of Decimal Numbers: %d\n",arr[4]);

printf("Count of HEXADECIMAL: %d\n",arr[5]);

printf("Count of Octal: %d\n",arr[6]);

printf("Count of Real: %d\n",arr[7]);

printf("Count of character literals: %d\n",arr[8]);

printf("Count of Struct: %d\n",arr[9]);

printf("Count of Union: %d\n",arr[10]);

printf("Count of Conditionals: %d\n",arr[11]);

printf("Count of Jumps: %d\n",arr[12]);

```

printf("Count of Loops: %d\n",arr[13]);
printf("\n");
return 0;
}

```

```

//input file
#include <stdio.h>
#define PRIME 31
int iter = 111;
void main()
{ unsigned long high = 0xffff2000;
float x = 0.2e-1;
struct { short age; char gender; float ht;
} person;
for (short k = 0; k < iter; k++)
{ if (k > 1 && k <= PRIME)
{ high = high >> 2;}
else
while (x != 1.0) x += 0.1;
}

```

```

//output
Keyword #include
Count of preprocessor derivatives: 1
Keyword #define
Count of preprocessor derivatives: 2
Keyword int
Identifier =PRIME
Count of Identifiers: 1
Count of Keywords: 3
Keyword void
Count of Keywords: 4
Keyword unsigned
Keyword long
Identifier =high
Count of Identifiers: 2
Keyword float
Identifier =x
Count of Identifiers: 3

```

```

Count of Keywords: 6
Keyword struct
Count of Keywords: 7
Delimiter {
Count of Delimiters: 1
Keyword short
Identifier =age
Count of Identifiers: 4
Keyword char
Identifier =gender
Count of Identifiers: 5
Keyword float
Identifier =ht
Count of Identifiers: 6
Delimiter ;
Count of Delimiters: 2
Delimiter }
Count of Delimiters: 3

```

Keyword for loop
Count of Loops: 1
Delimiter (
Count of Delimiters: 4
Keyword short
Identifier =k
Count of Identifiers: 7
Operator <
Count of Operators: 1
Identifier =iter
Count of Identifiers: 8
Delimiter ;
Count of Delimiters: 5
Identifier =k
Count of Identifiers: 9
Operator >
Count of Operators: 2
Numeric value
Count of Numeric values: 1
Operator &&
Count of Operators: 3
Identifier =PRIME
Count of Identifiers: 10
Delimiter)
Count of Delimiters: 6
Delimiter {
Count of Delimiters: 7
Identifier =high
Count of Identifiers: 11
Operator >>
Count of Operators: 4
Numeric value

Count of Numeric values: 2
Delimiter ;
Count of Delimiters: 8
Delimiter }
Count of Delimiters: 9
Keyword else
Count of Keywords: 8
Keyword while loop
Count of Loops: 2
Delimiter (
Count of Delimiters: 10
Identifier =x
Count of Identifiers: 12
Operator !=
Count of Operators: 5
Numeric value
Count of Numeric values: 3
Operator ++
Count of Operators: 6
Delimiter)
Count of Delimiters: 11
Delimiter ;
Count of Delimiters: 12
Delimiter }
Count of Delimiters: 13

Count of preprocessor derivatives: 2
Count of Identifiers: 12
Count of Keywords: 8
Count of Operators: 6
Count of Numeric values: 3
Count of Delimiters: 13

Lab Assignment-6

Feb 15, 2024

Objective : To write a lexical analyzer for complete C language including comments and string literals.

P1. This experiment mainly concerns with adding the following 3 features to your existing lex scripts.

- Add a regex to recognize single line comments in C {line starting with with “//”)
- Add a regex to recognize multiple line comments in C {starts with the symbols “/*” and ends with “*/” and can span multiple lines.
- Add a regex to detect string literals (starts with “ and ends with “)

Test your generated scanner with sample inputs generated by you that contain all the features

including the patterns used by you earlier such as identifiers, numbers, operators, delimiters, Etc.

Lex script:

```
%{
#include <stdio.h>
int lineno;
%}
%option noyywrap
D [0-9]
L [a-zA-Z]
O [0-7]
H [0-9a-fA-F]
%%
"/".* {printf("Single line comment \n");}
"/*"(.*)["*/"] {printf("Multiline comment \n");}
\"(\\.|[^\"])*\" {printf("String literal: %s\n", yytext);}
[\t]+
int|main|float|char|double|for|if|while|else|goto|return|continue|float|char|const|
static|do|void {printf("Keyword: %s\n",yytext);}
{L}({L}|{D})* {printf("Variable: %s\n",yytext);}
"/" {printf("DIV \n");}
"+" {printf("ADD \n");}
"-" {printf("MINUS \n");}
"="|"+="|"==" {printf("ASSIGNOP: %s \n",yytext);}
">" {printf("GREATER THAN \n");}
"<=" {printf("LESS THAN OR EQUAL TO\n");}
```

```

"|"{"| "[" {printf("LEFT PAR: %s\n",yytext);}
")|"}"| "]" {printf("RIGHT PAR: %s\n",yytext);}
";"|"," {printf("Delimiter: %s\n",yytext);}
"<"| ">" {printf("Bitwise Shift\n");}
"0x"{H}+ {printf("Hexadecimal number\n");}
"0"{O}+ {printf("Octal number\n");}
{D}*"."{D}* {printf("Floating point value: %s\n",yytext);}
{D}+ {printf("Numeric value: %s\n",yytext);}
\n {}
%%
int main(int argv , char *argc[]){
    yyin=fopen(argc[1],"r");
    yylex();
    fclose(yyin);
    return 0;
}

```

```

//output
#Variable: include
<Variable: stdio
Floating point value: .
Variable: h
GREATER THAN

```

```

Keyword: int
Keyword: main
LEFT PAR: (
RIGHT PAR: )
LEFT PAR: {

```

```

Single line comment
Keyword: int
Variable: x
ASSIGNOP: =
Numeric value: 10
Delimiter: ;
Single line comment

```

```

Multiline comment

```

```

Keyword: float
Variable: y
ASSIGNOP: =
Floating point value: 3.14
Delimiter: ;
Single line comment
Keyword: int
Variable: z
ASSIGNOP: =
Hexadecimal number
Delimiter: ;
Single line comment
Keyword: int
Variable: octal
ASSIGNOP: =
Octal number
Delimiter: ;
Single line comment
Variable: printf
LEFT PAR: (
String literal: "x = %d\n"
Delimiter: ,
Variable: x

```

RIGHT PAR:)

Delimiter: ;

Variable: printf

LEFT PAR: (

String literal: "y = %f\n"

Delimiter: ,

Variable: y

RIGHT PAR:)

Delimiter: ;

Variable: printf

LEFT PAR: (

String literal: "z = %d\n"

Delimiter: ,

Variable: z

RIGHT PAR:)

Delimiter: ;

Variable: printf

LEFT PAR: (

String literal: "octal = %d\n"

Delimiter: ,

Variable: octal

RIGHT PAR:)

Delimiter: ;

Single line comment

Keyword: int

Variable: a

ASSIGNOP: =

Variable: x

ADD

Numeric value: 5

Delimiter: ;

Single line comment

Keyword: int

Variable: b

ASSIGNOP: =

Variable: x

MINUS

Numeric value: 3

Delimiter: ;

Single line comment

Keyword: float

Variable: c

ASSIGNOP: =

Variable: y

* Numeric value: 2

Delimiter: ;

Single line comment

Keyword: float

Variable: d

ASSIGNOP: =

Variable: y

DIV

Numeric value: 2

Delimiter: ;

Single line comment

Keyword: int

Variable: e

ASSIGNOP: =

Variable: x

% Numeric value: 3

Delimiter: ;

Single line comment

Variable: printf

LEFT PAR: (

String literal: "a = %d\n"

Delimiter: ,

Variable: a

RIGHT PAR:)

Delimiter: ;

Variable: printf

LEFT PAR: (

String literal: "b = %d\n"

Delimiter: ,

Variable: b

RIGHT PAR:)

Delimiter: ;

Variable: printf
LEFT PAR: (
String literal: "c = %f\n"
Delimiter: ,
Variable: c
RIGHT PAR:)
Delimiter: ;

Variable: printf
LEFT PAR: (
String literal: "d = %f\n"
Delimiter: ,
Variable: d
RIGHT PAR:)

Delimiter: ;

Variable: printf
LEFT PAR: (
String literal: "e = %d\n"
Delimiter: ,
Variable: e
RIGHT PAR:)
Delimiter: ;

Keyword: return
Numeric value: 0
Delimiter: ;

RIGHT PAR: }

Lab Assignment-7

Feb 29, 2024

Objective : To write a lexical analyzer for X86-64 bit assembly language programs.

P1. The problem is to acquire skills to process assembly code of a contemporary real architecture, namely the X86-64 bit architecture, which most likely is the same as that of the server used for this course.

The task is to

- examine the assembly code generated by gcc,
- identify patterns in the assembly code of interest,
- write a lex script that identifies the patterns, and
- finally produces some statistical data about the various elements in the assembly code.

The problem specification is illustrated through an example. The C program, “prog.c”, is compiled using “\$ gcc prog.c -O2 S” to generate assembly code using level 2 optimization. Let the generated assembly code be named as “prog-opt.s”. Check the assembly code on your server as it may be slightly different than that given

below.

// C program named as prog.c

```
int main()
{ int a[1000], i, j;
  int sum = 10000;
  for (i = 0; i < 100; i++) a[i] = i;
  for (i = 0; i < 10; i++)
  for (j = 0; j < i*i; j++) a[i] = a[i]*a[j];
  printf(" sum : %d \n", sum);
  return 0;
}
```

The generated assembly code and the desired output after processing the assembly code are given in the following table.

X86-64 bit assembly program "prog-opt.s"	Desired Output after Processing "prog-opt.s"
<pre> .file "prog.c" .text .section .rodata.str1.1,"aMS",@progbits,1 .LC0: .string " sum : %d \n" .section .text.startup,"ax",@progbits .p2align 4 .globl main .type main, @function main: .LFB23: .cfi_startproc endbr64 subq \$8, %rsp .cfi_def_cfa_offset 16 movl \$10000, %edx movl \$1, %edi xorl %eax, %eax leaq .LC0(%rip), %rsi call __printf_chk@PLT xorl %eax, %eax addq \$8, %rsp .cfi_def_cfa_offset 8 ret .cfi_endproc .LFE23: .size main, .-main .ident "GCC: (Ubuntu 9.4.0-1ubuntu1~20.04.2) 9.4.0" .section .note.GNU-stack,"",@progbits .section .note.gnu.property,"a" .align 8 .long 1f - 0f .long 4f - 1f .long 5 0: </pre>	<pre> C Source File : .file "prog.c" Assembler Directive : .text Assembler Directive : .section .rodata.str1.1,"aMS",@progbits,1 Label : .LC0 DELIMITER : Format String : .string " sum : %d \n" Assembler Directive : .section .text.startup,"ax",@progbits Assembler Directive : .p2align 4 Assembler Directive : .globl main Assembler Directive : .type main, @function Identifier : main DELIMITER : Label : .LFB23 DELIMITER : Assembler Directive : .cfi_startproc Other Instructions : endbr64 Quad Word Instructions : subq Immediate operand : \$8 DELIMITER , 64 bit Register operand : %rsp Assembler Directive : .cfi_def_cfa_offset 16 Long Word Instructions : movl Immediate operand : \$10000 DELIMITER , 32 bit Register operand : %edx Long Word Instructions : movl Immediate operand : \$1 DELIMITER , 32 bit Register operand : %edi Long Word Instructions : xorl 32 bit Register operand : %eax DELIMITER , 32 bit Register operand : %eax Quad Word Instructions : leaq Label : .LC0 LEFT PAR : (</pre>

X86-64 bit assembly program "prog-opt.s"	Desired Output after Processing "prog-opt.s"
<pre> .string "GNU" 1: .align 8 .long 0xc0000002 .long 3f - 2f 2: .long 0x3 3: .align 8 4: </pre>	<pre> 64 bit Register operand : %rip RIGHT PAR :) DELIMITER , 64 bit Register operand : %rsi Other Instructions : call Function call : __printf_chk@PLT Long Word Instructions : xorl 32 bit Register operand : %eax DELIMITER , 32 bit Register operand : %eax Quad Word Instructions : addq Immediate operand : \$8 DELIMITER , 64 bit Register operand : %rsp Assembler Directive : .cfi_def_cfa_offset 8 Identifier : ret Assembler Directive : .cfi_endproc Label : .LFE23 DELIMITER : Assembler Directive : .size main, -main Assembler Directive : .ident "GCC: (Ubuntu 9.4.0 1ubuntu1~20.04.2) 9.4.0" Assembler Directive : .section .note.GNU-stack,"",@progbits Assembler Directive : .section .note.gnu.property,"a" Assembler Directive : .align 8 Assembler Directive : .long 1f - 0f Assembler Directive : .long 4f - 1f Assembler Directive : .long 5 Number : 0 DELIMITER : Format String : .string "GNU" Number : 1 DELIMITER : Assembler Directive : .align 8 Assembler Directive : .long 0xc0000002 Assembler Directive : .long 3f - 2f Number : 2 DELIMITER : Assembler Directive : .long 0x3 Number : 3 DELIMITER : Assembler Directive : .align 8 Number : 4 DELIMITER : No of Assembler Directives: 23 No of in-line comment : 0 No of Strings : 0 Count of Immediate Operands Used 4 Count of 64 bit Registers Used 4 Count of 32 bit Registers Used 6 Count of Byte length Instructions Used 0 Count of Word length Instructions Used 0 Count of Long Word length Instructions Used 4 Count of Quad length Instructions Used 3 </pre>

X86-64 bit assembly program "prog-opt.s"	Desired Output after Processing "prog-opt.s"
	Count of Stack Instructions Used 0

```
//Lex script
%{
#include <stdio.h>
int lineno;
int cnt[18];
}%
%option noyywrap
P mov|sub|xor|lea|add|ret|cmp|imul|sal|shr|and|or|not|clt
A [a-zA-Z]
X [a-zA-Z0-9]
O [0-7]
H [0-9a-fA-F]
D [0-9]
%%
[\t]+
"%r"^[^\\t]* {printf("64 bit regiter: %s\n",yytext); cnt[0]++;}
"%e"^[^\\t]* {printf("32 bit register: %s\n",yytext); cnt[1]++;}
{P} {printf("Common instruction op codes: %s\n",yytext); cnt[2]++;}
{P}"l" {printf("Long word instructions: %s\n",yytext); cnt[3]++;}
{P}"q" {printf("Quad word instructions: %s\n",yytext); cnt[4]++;}
{P}"w" {printf("Short word instructions: %s\n",yytext); cnt[5]++;}
endbr64|call|leave {printf("Other assembly instructions: %s\n",yytext); cnt[6]++;}
jmp|je|jne|js|jns|jg|jge|jl|jle {printf("Jump instructions: %s\n",yytext); cnt[7]++;}
pushq|popq {printf("Stack Instructions: %s\n",yytext); cnt[8]++;}
".file"(.)* {printf("C source file: %s\n",yytext); cnt[9]++;}
".string"(.)* {printf("Format string: %s\n",yytext); cnt[10]++;}
".L"{X}(.)* {printf("Label: %s\n",yytext); cnt[11]++;}
".("(.)* {printf("Assembler Directives: %s\n",yytext); cnt[12]++;}
"${X}* {printf("operand: %s\n",yytext); cnt[13]++;}
":|'|"|"('|")" {printf("Delimiter: %s\n",yytext); cnt[14]++;}
(-)?{D}* {printf("Real: %s\n",yytext); cnt[15]++;}
{A}{X}* {printf("Identifier: %s\n",yytext); cnt[16]++;}
[\n]+
%%
int main(int argv , char *argc[]){
    for(int i=0;i<18;i++) cnt[i]=0;
    yyin = fopen(argc[1],"r");
    yylex();
    fclose(yyin);
    printf("number of 64 bit registers: %d\n",cnt[0]);
```

```

printf("number of 32 bit registers: %d\n",cnt[1]);
printf("number of Common word instructions: %d\n",cnt[2]);
printf("number of Long word instructions: %d\n",cnt[3]);
printf("number of Quad word instructions: %d\n",cnt[4]);
printf("number of Short word instructions: %d\n",cnt[5]);
printf("number of other assembly instructions: %d\n",cnt[6]);
printf("number of jump instructions: %d\n",cnt[7]);
printf("number of stack instructions: %d\n",cnt[8]);
printf("number of Labels: %d\n",cnt[11]);
printf("number of Format strings: %d\n",cnt[10]);
printf("number of Assembler directives: %d\n",cnt[12]);
printf("number of operands: %d\n",cnt[13]);
printf("number of delimiters: %d\n",cnt[14]);
printf("number of Reals: %d\n",cnt[15]);
printf("number of identifiers: %d\n",cnt[16]);
return 0;
}

```

<pre> //output .file "prog.c" .text .section .rodata.str1.1,"aMS",@progbits,1 .LC0: .string " sum : %d \n" .section .text.startup,"ax",@progbits .p2align 4 .globl main .type main, @function main: .LFB23: .cfi_startproc endbr64 subq \$8, %rsp .cfi_def_cfa_offset 16 movl \$10000, %edx movl \$1, %edi xorl %eax, %eax leaq .LC0(%rip), %rsi call __printf_chk@PLT xorl %eax, %eax addq \$8, %rsp .cfi_def_cfa_offset 8 </pre>	<pre> ret .cfi_endproc .LFE23: .size main, .-main .ident "GCC: (Ubuntu 9.4.0-1ubuntu1~20.04.2) 9.4.0" .section .note.GNU-stack,"",@progbits .section .note.gnu.property,"a" .align 8 .long 1f - 0f .long 4f - 1f .long 5 0: .string "GNU" 1: .align 8 .long 0xc0000002 .long 3f - 2f 2: .long 0x3 3: .align 8 4 </pre>
--	--

Lab Assignment-8

Mar 07, 2024

Objective : to use YACC to generate a LALR(1) parser and experiment with its performance.

P1. Two sample files, “pointer.l” and “pointer.y” are given to you, the former is a lex script (specifying regexes)

and the latter is a yacc script. Read the two files and note the difference in this lex script and the earlier ones that

we have used so far. Generate a parser, given by “parser1” by executing the following commands.

- Option “-v” in yacc stands for verbose; in particular produces a file named as “y.output” which is a

verbose description of the generated LALR(1) parser.

- Option “-d” produce a header file, y.tab.h, which contains definitions that enables communication

between the scanner and the parser, in particular between yylex() and yyparse().

```
$ lex pointer.l
```

```
$ yacc -d -v pointer.y or using $yacc -dv pointer.y
```

```
$ gcc lex.yy.c y.tab.c -ll -o parser1
```

(a) Test the parser with a few inputs provided in the following table. Explain the output with respect to

the scripts. For example, the output for the first 2 inputs versus the 3rd input.

input1

****a=*a**

input2

a = *a

input3

****a = *a = a**

```
%{
#include <stdio.h>
#include "y.tab.h"
%}
/* Minimal lex script to return the tokens
of interest */
%%
```

```
"" {return ('*');}
"=" {return ('=');}
"a" {return ('a');}
```

```
%%
int yywrap(void) {return 1;}
```

```
/*C declarations*/
```

```
%{
#include <stdio.h>
int yylex(void);
void yyerror(char *);
%}
```

```
/* YACC Declarations */
```

```
%token a
%right '='
%right '*'
```

```
/* Grammar follows */  
%%
```

```
S:    L '=' R  
      | R  
      ;  
L:    '*' R  
      | 'a'  
      ;  
R:    L
```

```
;
```

```
%%  
void yyerror(char *text) {printf("%s\n",  
    text); }
```

```
int main ()  
{yyparse (); return 0;}
```

//output:

```
(hashat@Kali)-[~/cd/prac/l8]  
$ ./p1  
**a=*a  
a=*a  
syntax error  
  
(hashat@Kali)-[~/cd/prac/l8]  
$ ./p1  
**a=*a=a  
syntax error
```


Objective : Communication between scanner and parser; simple applications of semantic analysis in the domain of compilers.

- P1.** The objective of this problem is to understand the role played by the “return X “ statements used in lex and their exact role in the yacc script. Read the contents of the 2 files, “p1.l” and “p1.y” to get a feel of the intention encoded therein. Generate a parser, say p1, and run the executable on the input shown below. Document your finding in a text file. Input : 20, 3 , 5 , 11
- P2.** The objective of this problem is to enable communication of values between scanner and the parser. P1 is now changed by modifying the lex script to the file, “p2.l”, while the yacc script is same, “p1.y”. Redo the task stated in P1, generate a new parser, say p2, and find the output for the same input.
- P3.** The objective of this problem is to perform attribute evaluation in the yacc script. We wish to design a simple calculator involving unsigned numbers and two operations, {+, *}. The calculator must honour all the properties of these 2 operators as given by language C. Read the contents of 2 files, “p3.l” and “p3.y”, understand the code placed therein to generate an executable, say calc1. Run the executable on the input “20 + 3 * 4 + 6 * 2”
- P4.** The grammar used in P3 is unambiguous, it used the empirical rules that satisfy the properties of both the operators. The same calculator is now designed using an ambiguous grammar but the uses the special features of yacc that can handle ambiguity. Read and understand the contents of the file, “p4.y”, and identify the yacc features used to preserve the operator properties. Generate a new calculator, using “p3.l” and “p4.y”, say calc2. Run calc2 on the same input as that of P3, observe the output
- P5.** The objective is to communicate multiple attributes between scanner and parser. Read the contents of the files, “lex-yacc-attr.l” and “lex-yacc-attr.y” and understand the codes therein. Make a note of the features used for communication of multiple attributes. Generate a parser and test its performance with the input :count, intsum, floatsum, mean, mode, median Test the performance with a few sample inputs generated by you and report your findings in the document.

P1.-

//LEX FILE

%{

/* Illustrates communication of string b/w scanner and parser

Note the union declaration in yacc script which yylval to

either an int or a char*;

Since yylval is not its default value, yylval cannot be assigned

directly, we have to assign yylval.name to send a string

or yylval.numval to send an int value. */

#include <stdio.h>

#include <stdlib.h>

#include "y.tab.h"

// int yylval;

%}

ws [\t\n]+

letter [A-Za-z]

digit [0-9]

id {letter}({letter}|{digit})*

%%

{ws} {; /* No action */}

{digit}+ {return NUM;}

", " {return ',';}

%%

int yywrap(void)

{ return 1; }

//YACC FILE

/*C declarations*/

/* Partner of lex-yacc-comm.l script which uses an union

//INPUT : 20,3,5,11

*/

%{ #include <stdio.h>

#include "y.tab.h"

int yylex(void);

void yyerror(char*);

extern int yylineno, yychar;

%}

/* YACC Declarations */

%token NUM

%type list

%%

list : list ',' NUM

{ printf("Rule list -> list , NUM : read

Identifier : %d and \$\$ = %d\n", \$3, \$\$);

\$\$ = \$1;

}

| NUM { printf (" term -> NUM : %d %d\n", \$1, \$\$);}

;

%%

void yyerror(char *s)

{printf(" %s line number : %d near symbol %c\n", s, yylineno, (char)yychar);}

int main ()

{ // yydebug = 1;

yyparse ();

}


```

%%
{ws}      { /* No action */}
{digit}+  {yylval=atoi(yytext); return NUM;}
", "      {return ',';}
%%
int yywrap(void)
{ return 1; }
//OUTPUT:

```

```

term -> NUM : 20
Rule list -> list , NUM : read Identifier : 3 and $$ = 20
term -> NUM : 3
Rule list -> list , NUM : read Identifier : 5 and $$ = 20
term -> NUM : 5
Rule list -> list , NUM : read Identifier : 11 and $$ = 20
term -> NUM : 11

```

P3.- p3.l

/* lex script to recognize unsigned integers
and a few operators */

```

%{
#include <stdlib.h> /* why ?? */
void yyerror(char*);
#include "y.tab.h"
int yylval;
%}
ws [ \t\n]+
digit [0-9]
%%
{ws}      ;
{digit}+ { yylval = atoi(yytext); return
NUM;}
"+"      {return '+';}
"*"      {return '*'}
.        {yyerror("invalid character\n");}
%%

```

```

int yywrap(void)
{ return 1; }
//p3.y
%{
#include <stdio.h>
int yylex(void);
void yyerror(char *);
%}

```

```

%token NUM
%%

```

```

P:
  P E  {printf("Value : %d\n", $2);}
  |
  ;
E: T  {printf(" Value : %d\n", $1); $$ = $1;}
  | E '+' T  {$$ = $1+$3; printf("%d + %d val
= %d \n", $1, $3, $$);}
  ;

```

```

T: T '*' NUM {$$ = $1 * $3; printf("%d * %d
val = %d \n", $1, $3, $$);}
| NUM {printf(" Integer : %d\n", $1);
$$=$1;}
;
%%

void yyerror (char* mess)
//output:

```

```

{ fprintf(stderr, "%s\n", mess); }

int main()
{
    yyparse();
    return 0;
}

```

```

Integer : 20
Integer : 3
Integer : 4
3 * 4 val = 12
Integer : 6
Integer : 2
6 * 2 val = 12
20 + 12 val = 32
32 + 12 val = 44
Value : 44

```

P4.- //p4.y

```

%{
#include <stdio.h>
int yylex(void);
void yyerror(char *);
}%
%token NUM
%left '+' '-'
%left '*' '/'
%%

P:
P E {printf("Value : %d\n", $2);}
|
;

```

E:

```
NUM  {$$ = $1; printf(" Integer : %d\n", $$);}
| E '+' E {$$ = $1 + $3; printf("%d + %d val = %d \n", $1, $3, $$);}
| E '*' E {$$ = $1 * $3; printf("%d * %d val = %d \n", $1, $3, $$);}
;
%%
void yyerror (char* mess)
{ fprintf(stderr, "%s\n", mess); }
int main()
{
    yyparse();
    return 0;
}
//output:
```

```
Integer : 20
Integer : 3
Integer : 4
3 * 4 val = 12
Integer : 6
Integer : 2
6 * 2 val = 12
20 + 12 val = 32
32 + 12 val = 44
Value : 44
```

P5.

```
//lex-aatr-yacc.l
%{
#include <stdio.h>
#include <stdlib.h>
#include "y.tab.h"
%}
ws [ \t\n]+
letter [A-Za-z]
digit [0-9]
id {letter}({letter}|{digit})*

%{
{ws}      {; /* No action */}
{id}      {yylval.threeattr.name=yytext;
           yyval.threeattr.len=yytext;
           yyval.threeattr.code=1;
           return ID;
           }
", "      {return ',';}
%%

int yywrap(void)
{ return 1; }
```

```
//lex-attr-yacc.y
/*C declarations*/
%{ #include <stdio.h>
#include "y.tab.h"
int yylex(void);
void yyerror(char*);
extern int yylineno, yychar;
%}

/* YACC Declarations */
%union
{ int numval;
struct attr
{ char * name;
int len;
int code;
} threeattr;
}
%token <threeattr> ID
%type <numval> list
%%
```

```
list : list ',' ID
      { printf (" Rule 1 used \n");
        printf(" Attributes of ID : ( %s, %d, %d)
\n", $3.name, $3.len, $3.code);
        $1 = $3.code;
      }

      | ID { printf (" Rule 2 used \n");
              printf(" Attributes of ID : (%s, %d,
%d) \n", $1.name, $1.len, $1.code);
              $$ = $1.code;
            }
      ;

%%
void yyerror(char *s)
{printf(" %s line number : %d near symbol %c
\n", s, yylineno, (char)yychar);}

int main ()
{ // yydebug = 1;
  yyparse ();
```

Input: count,intsum,floatsum,mean,mode,median
output:

```
Rule 2 used
Attributes of ID : (count, 5, 1)
Rule 1 used
Attributes of ID : (intsum, 6, 1)
Rule 1 used
Attributes of ID : (floatsum, 8, 1)
Rule 1 used
Attributes of ID : (mean, 4, 1)
Rule 1 used
Attributes of ID : (mode, 4, 1)
Rule 1 used
Attributes of ID : (median, 6, 1)
```

P6.to design a comprehensive calculator that can process an input of the form :
 $20 + 3 * 2.5 * 2 + 5.5$
 and produce output of the form sown below.

Int : 20

value is int 20

Int Value : 20

Int : 3

value is int 3

Real : 2.500000

$3 * 2.500000 = 7.500000$

Int : 2

$7.500000 * 2 = 15.000000$

$20 * 15.000000 = 35.000000$

Real : 5.500000

value is float 5.500000

$35.000000 * 5.500000 = 40.500000$

Final Real Value : 40.500000

-

//p6.l

%{

#include <stdio.h>

#include <stdlib.h>

#include "y.tab.h"

%}

%union {

int intval;

float realval;

}

%token <intval> INTEGER

%token <realval> REAL

%token PLUS MULTIPLY

%%

[\t\n] ; /* skip whitespace */

[0-9]+ {

yyval.intval = atoi(yytext);

return INTEGER;

}

[0-9]+ "." [0-9]* {

yyval.realval = atof(yytext);

return REAL;

}

"+" { return PLUS; }

"*" { return MULTIPLY; }

. { fprintf(stderr, "Invalid character:
%s\n", yytext); }

%%

int yywrap() {

return 1;

}


```

//p6.y
%{
#include <stdio.h>
#include <stdlib.h>
%}

%union {
    int intval;
    float realval;
}

%token <intval> INTEGER
%token <realval> REAL
%token PLUS MULTIPLY

%type <intval> expression_int
%type <realval> expression_real

%%

expression_int: INTEGER {
    printf("Int : %d\n", $1);
    printf("Value is int %d\n", $1);
    $$ = $1;
}
| expression_int PLUS expression_int {
    printf("Int : %d\n", $1);
    printf("Int : %d\n", $3);
    printf("Int Value : %d\n", $1);
    $$ = $1 + $3;
    printf("Int Value : %d\n", $$);
}
| expression_real MULTIPLY expression_int
{
    printf("Real : %f\n", $1);
    printf("%f * %d = %f\n", $1, $3, $1 * $3);
    $$ = $1 * $3;
}
| INTEGER MULTIPLY expression_real {
    printf("Int : %d\n", $1);
    printf("%d * %f = %f\n", $1, $3, $1 * $3);
}

```

```

    $$ = $1 * $3;
}
| REAL PLUS expression_int {
    printf("Real : %f\n", $1);
    printf("Value is float %f\n", $1);
    printf("%f + %d = %f\n", $1, $3, $1 + $3);
    $$ = $1 + $3;
}
| REAL MULTIPLY expression_real {
    printf("Real : %f\n", $1);
    printf("%f * %f = %f\n", $1, $3, $1 * $3);
    $$ = $1 * $3;
}
;

expression_real: REAL {
    printf("Real : %f\n", $1);
    printf("Value is float %f\n", $1);
    $$ = $1;
}
| expression_real PLUS expression_real {
    printf("Real : %f\n", $1);
    printf("Real : %f\n", $3);
    printf("Value is float %f\n", $1);
    $$ = $1 + $3;
}
| expression_real MULTIPLY expression_real
{
    printf("Real : %f\n", $1);
    printf("%f * %f = %f\n", $1, $3, $1 * $3);
    $$ = $1 * $3;
}
;
%%

int main() {
    yyparse();
    return 0;
}

void yyerror(const char *s) {
    fprintf(stderr, "Error: %s\n", s);
}

```

//input: $20 + 3 * 2.5 * 2 + 5.5$

//output:

Int : 20

value is int 20

Int Value : 20

Int : 3

value is int 3

Real : 2.500000

$3 * 2.500000 = 7.500000$

Int : 2

$7.500000 * 2 = 15.000000$

$20 * 15.000000 = 35.000000$

Real : 5.500000

value is float 5.500000

$35.000000 * 5.500000 = 40.500000$

Final Real Value : 40.500000

Lab Assignment-10

Mar 21, 2024

Objective : Communication between scanner and parser; simple applications of semantic analysis in the domain of compilers.

Grammar for Declarations in a scope	Remarks
R1. $P \rightarrow dlist$ R2. $dlist \rightarrow dlist\ decl;$ R3. $dlist \rightarrow decl;$ R4. $decl \rightarrow type\ id$ R5. $decl \rightarrow decl,\ id$ R6-R10. $type \rightarrow char\ \ short\ \ int\ \ long\ \ double$	Rules R4 and R5 are also commonly written as without changing the underlying grammar $decl \rightarrow type\ list$ $list \rightarrow list,\ id\ \ id$ type has minimally 2 attributes, type.width and type.lexeme which denote the size in bytes of "type.lexeme"; id has the attribute "id.name"

4. The sizes in bytes of basic data types on my system are given below.

char	short	int	long	long long	float	double	long double
1	2	4	8	8	4	8	16

Write a C program to determine the sizes on your server and use them in your SDTS.

5. A template lex and yacc script that illustrates how to pass multiple attributes between the scanner and parser in a general setting are given in the files, "lex-yacc-attr.l", "lex-yacc-attr.y" and a sample input, "listinp". You may generate a parser and check its behaviour on the sample input.

P1. The objective of this problem is to write a grammar to generate a list of declarations. Then add semantic rules to your grammar so that the SDTS can process input declarations and generate output of the kind shown below. Note that you need to write a line of output for each declared symbol in the format given. The offset is a relative address of the symbol. For example, "a" being the first symbol is assigned $offset(a)=0$, while $offset(sum) = offset(a) + sizeof(a) = 4$ bytes, and so on. The size of all the declarations processed is calculated and displayed on the last line. Note that in this implementation, your parser is not capable of handling errors in declarations.

Input	Desired Output
int a, sum, coefficient; char ch1, ch2, ch3; float fl1, fl2, fl3;	Name a Type int Width 4 Offset 0 Name sum Type int Width 4 Offset 4 Name coefficient Type int Width 4 Offset 8 1 declaration seen Name ch1 Type char Width 1 Offset 12 Name ch2 Type char Width 1 Offset 13 Name ch3 Type char Width 1 Offset 14 2 declarations seen Name fl1 Type float Width 4 Offset 15 Name fl2 Type float Width 4 Offset 19

```

//lex file%{
#include <stdio.h>
#include "y.tab.h"
%}
%%

[a-zA-Z]+    { yylval.strval = yytext; return NAME; }
[int]+       { return INT; }
[char]+      { return CHAR; }
[float]+     { return FLOAT; }
[0-9]+       { yylval.intval = atoi(yytext); return NUMBER; }
[,]          { return ','; }
[ \t\n]      ; /* skip whitespace */
.            { return yytext[0]; }

%%

int yywrap() {
    return 1;
}

//yacc file
%{
#include <stdio.h>
#include <string.h>
%}

union {
    char *strval;
    int intval;
}

%token <strval> NAME
%token <intval> INT CHAR FLOAT NUMBER

%%

declaration_list: declaration_list declaration
                | declaration
                ;

declaration: type_specifier variable_list ';' {
    printf("Name %s Type %s Width %d Offset\n", $2.name, $1, $2.width, $2.offset);
    $$ = $<intval>3 + 1;
    if ($$ == 1)
        printf("%d declaration seen\n", $$);
    else
        printf("%d declarations seen\n", $$);
}

type_specifier: INT { $$ = "int"; }
              | CHAR { $$ = "char"; }
              | FLOAT { $$ = "float"; }
              ;

variable_list: variable
              | variable_list ',' variable
              ;

variable: NAME {
    $$ = $1;
    strcpy($$.name, $1);
    $$.width = 4;
    $$.offset = $<intval>3;
    $<intval>3 += $<intval>5;
}
;

%%

```

```
int main() {  
    yyparse();  
    return 0;  
}
```

```
void yyerror(const char *s) {  
    fprintf(stderr, "Error: %s\n", s);  
}
```

Input:

```
int a, sum, coefficient;  
char ch1, ch2, ch3;  
float fl1, fl2, fl3;
```

Output:

```
Name a Type int Width 4 Offset 0  
Name sum Type int Width 4 Offset 4  
Name coefficient Type int Width 4 Offset 8  
3 declarations seen  
Name ch1 Type char Width 1 Offset 12  
Name ch2 Type char Width 1 Offset 13  
Name ch3 Type char Width 1 Offset 14  
6 declarations seen  
Name fl1 Type float Width 4 Offset 15  
Name fl2 Type float Width 4 Offset 19  
Name fl3 Type float Width 4 Offset 23  
9 declarations seen
```

